

第6章_中央处理器_知识点整理

第6章 中央处理器

关联：本章是计算机组成原理的核心章节，与第5章（指令系统）紧密衔接——指令系统定义了CPU能执行什么指令，本章则描述CPU如何执行这些指令。第7章（指令流水线）是本章多周期处理器设计的进一步性能优化。数据通路的设计也与第2章（数据信息的表示）和第3章（运算方法与运算器）相关。

6.1 中央处理器概述

6.1.1 中央处理器的功能

CPU（Central Processing Unit）：由运算器和控制器组成，是计算机的核心。

CPU的主要功能是执行程序；计算机上电后（或者按Reset键后），CPU即开始周而复始地进行取指令和执行指令的工作。

CPU应具备的五大功能：

功能	说明
程序控制	控制程序中的指令执行顺序（包括顺序执行、转移、子程序调用/返回等），正确确定下条指令的地址
操作控制	产生指令执行过程中需要的操作控制信号；如执行加法指令时，生成运算选择信号以保证进行加法操作
时序控制	对每个操作控制信号进行定时，按规定的时间顺序执行各操作
数据加工	对数据进行算术、逻辑运算，或将数据在各部件之间传送
中断处理	及时响应内部异常和外部中断请求

6.1.2 中央处理器的组成

早期计算机的CPU主要包括运算器和控制器；现代计算机的CPU还增加了Cache（L1 cache，内部cache）、MMU（存储器管理单元）、浮点运算器等单元。

图片信息：PPT第5页，图6.1 CPU的基本组成框图

1、CPU中的主要寄存器

寄存器	英文	说明
程序计数器	PC (Program Counter), x86称IP	保存将要执行的指令的字节地址
存储器地址寄存器	AR/MAR	保存CPU访问主存的单元地址
存储器数据寄存器	DR/MDR	存放从主存读出的数据或准备写入主存的数据
指令寄存器	IR	保存当前正在执行的指令；操作码送指令译码器ID，地址码送地址生成逻辑
通用寄存器组	GR（寄存器堆/寄存器文件）	可作为ALU累加器、变址寄存器、基址寄存器、地址指针、数据缓冲器
程序状态字寄存器	PSW/PSR, x86称EFLAGS	保存条件标志（C进位、V溢出、S负数、Z零）及中断/系统状态信息

2、操作控制器

- 接收指令译码器送来的指令译码信号
- 与时序信号、条件及状态信息进行组合
- 形成各种具有严格时间先后顺序的操作控制信号（微操作控制信号序列）
- CPU执行指令的过程就是CPU控制信息流的过程，操作控制器产生的微操作控制信号序列就是**控制流**

3、时序产生器

- 指令执行过程中的所有操作都必须按一定次序执行
- 需要对完成指令而执行的微操作控制信号进行时间调制
- 严格规定各信号的产生时间和持续时间（详见6.4节）

控制器分类：

控制器类型	实现技术	时序类型
硬布线控制器（6.5节）	时序逻辑技术	硬时序
微程序控制器（6.6节）	程序存储逻辑技术	软时序

6.2 指令周期

6.2.1 指令执行的一般流程

图片信息：PPT第8页，图6.2 控制器执行指令的一般流程图

指令执行的基本流程包括取指令、译码、取操作数、执行运算、写回结果等步骤。

6.2.2 指令周期的基本概念

指令周期：将一条指令从取出到执行完成所需要的时间。

指令周期的划分：

划分方式	周期组成
粗分	取指周期 + 执行周期
细分	取指周期 + 译码/取操作数周期 + 执行周期 + 写回周期

1、取指周期

- 以PC内容为地址从主存取出指令
- 计算后续指令地址（PC的新值）
- 顺序执行：PC + 当前指令字节长度（如PC + 4）
- 分支指令：根据寻址方式、分支条件、目标地址计算PC新值

2、译码/取操作数周期

- 对IR中的指令字进行译码，识别指令类型
- 根据地址码生成操作数有效地址，访问寄存器或主存单元

间址周期：

- 当寻址方式为间接寻址时，需要额外访存才能得到操作数地址
- 例：MOV EAX, @2008H
 - 主存2008H中的内容为A0A0F000H
 - 主存A0A0F000H中的内容为5000H
 - 则操作数S = 5000H，执行后EAX = 5000H
 - 访问2008H的访存周期为间址周期
 - 访问A0A0F000H的访存周期为取操作数周期

图片信息：PPT第10页，间接寻址示例图（形式地址=2008H，有效地址=A0A0F000H）

3、执行周期

- 控制器向ALU及数据通路发出操作控制命令
- 对已取出的操作数进行加工处理
- 将处理状态信息记录在PSW中
- 程序出现分支时，计算分支目标地址

4、写回周期

- 将运算结果写回到目的寄存器或存储器中

其他周期：

周期	说明
中断周期	指令执行完毕，出现异常或外部中断请求时，CPU进入中断响应周期；硬件保存断点，将中断处理程序入口地址送入PC
总线周期	完成总线操作及总线控制权的转移
I/O周期	完成输入输出操作

指令周期与时钟的关系：

- 机器周期（CPU周期）：从主存取出一个存储字所需的最短时间
- 时钟周期（节拍）：最基本的定时单位

定长指令周期：所有指令的指令周期相同，机器周期数固定，节拍数固定（早期三级时序系统）。

变长指令周期：现代计算机普遍采用，不同指令的机器周期数可变，按时钟周期同步，机器周期概念逐渐消失。

图片信息：PPT第12页，图6.3 指令周期、机器周期与时钟周期的关系

6.2.3 寄存器传送语言

RTL（Register Transfer Language）：用来表示指令执行过程中的操作。

基本术语：

- 操作：功能部件级的动作
- 微操作：指令序列中最基本的、不可再分割的动作

图片信息：PPT第13页，图6.4 指令执行过程中的操作与微操作（寄存器A到寄存器B的数据传送电路）

RTL描述规则：

符号	含义
M[addr]	主存单元或内容；作为源操作数表示主存内容，作为目的操作数表示主存单元
R[i]	通用寄存器组中的第i号寄存器或内容
B ← A 或 A → B	数据传送；A为源操作数，B为目的操作数

符号	含义
$Xy:z$	寄存器X的第y位到第z位的数据字段（如R0:7）
SignExt(X)	将X带符号扩展到32位；如0123H $\rightarrow$ 00000123H, 8123H $\rightarrow$ FFFF8123H
{X,Y}	将X、Y的比特位连接在一起；如{10,11,011}=1011011

复合表示示例：

- $M[R[i]]$ ：以第i号寄存器内容作为地址的主存单元（寄存器间接寻址）
- $M[PC]$ ：程序计数器PC所指向的主存单元内容

6.3 数据通路及指令操作流程

6.3.1 数据通路模型与定时

数据通路：数据在各功能部件之间传送的路径。可用总线或专用通路两种方式构建。

功能部件分类：

类型	特点	示例
数据处理单元	组合逻辑电路，输出只与当前输入有关	ALU、符号扩展单元、译码器
状态存储单元	带有存储功能的单元	存储器、寄存器

图片信息：PPT第15页，Logisim中的数据处理单元和状态存储单元示例

数据通路模型：

图片信息：PPT第16页，图6.5 数据通路模型（a：一个数据处理单元连接两个状态单元；b：简化模型）

- 状态单元只有在写使能信号有效且时钟触发到来时，才能将输入数据写入

D触发器定时模型（3个关键时间）：

时间参数	说明
建立时间（Setup Time）	时钟Clk触发前（上跳沿），输入数据Data须稳定一段时间（ T_{setup} ），便于数据被正确采样
保持时间（Hold Time）	时钟Clk触发后（上跳沿），输入数据Data还须稳定一段时间（ T_{hold} ），便于数据被准确传输

时间参数	说明
触发器延迟 (Clk_to_Q)	从时钟Clk触发（上跳沿）到输出Q稳定的时间 ($T_{clk_to_q}$)

图片信息：PPT第17页，D触发器定时模型示意图

数据通路最小时钟周期：

$$T_{min_clk} > T_{clk_to_q} + T_{max} + T_{setup}$$

- T_{max} ：数据处理单元（组合逻辑）关键路径延迟（所有输出信号稳定的最大延迟）
- 否则数据处理单元的输出来不及送到寄存器B

图片信息：PPT第18页，图6.6 数据通路与时钟周期

保持时间违例：

$$T_{hold} < T_{clk_to_q} + T_{min}$$

- T_{min} ：数据处理单元最短路径延迟
- 若 $T_{hold} \geq T_{clk_to_q} + T_{min}$ ，旧数据还没写入完成，新数据又要写入，发生保持时间违例
- 即时钟周期不能太短（主频不能太高）

图片信息：PPT第19页，图6.7 数据通路的保持时间违例

6.3.2 单总线结构的数据通路

单总线结构：CPU中的运算器、控制器、寄存器堆通过一条内总线（CPU内总线）连接。

图片信息：PPT第20页，图6.8 单总线结构的计算机框图（32位数据位宽）

功能模块说明：

模块	输入来源	输出去向
PC	内总线	内总线
AR	内总线	存储器MEM
DR	内总线或存储器	内总线或存储器
X（暂存器）	内总线	ALU
Z（暂存器）	ALU	内总线
ALU	A端来自X，B端来自内总线	Z
PSW	ALU	操作控制器
IR	内总线	内总线和指令译码器ID
ID	IR	操作控制器

模块	输入来源	输出去向
Regs (寄存器堆)	内总线	内总线; R#为读出编号, W#为写入编号
MEM	地址由AR提供, 数据来自/送到DR	—

- 每个功能部件输出端采用三态门进行输出控制, 防止总线数据冲突

控制信号及其功能 (表6.1):

序号	控制信号	功能说明
1	PCin	控制PC接收内总线数据, 需配合时钟
2	PCout	控制PC向内总线输出数据
3	ARin	控制AR接收内总线数据, 需配合时钟
4	DRin	控制DR接收内总线数据, 需配合时钟
5	DRout	控制DR向内总线输出数据
6	DREin	控制DR接收从主存读出的数据, 需配合时钟
7	DREout	控制DR向主存输出数据
8	Xin	控制暂存器X接收内总线数据, 需配合时钟
9	+4	ALU的A端口数据加4输出
10	ADD	控制ALU执行加法 ($A + B$)
11	SUB	控制ALU执行减法 ($A - B$)
12	PSWin	控制PSW接收ALU运算状态, 需配合时钟
13	Zout	控制Z向内总线输出数据 (Z无输入控制, 每个时钟都锁存新值)
14	IRin	控制IR接收内总线数据, 需配合时钟
15	IR(A)out	控制IR中的分支目标地址输出到内总线
16	IR(I)out	控制IR中的立即数 (符号扩展为32位) 输出到内总线
17	Write	存储器写命令, 需配合时钟
18	Read	存储器读命令
19	Rin	控制寄存器堆接收内总线数据, 需配合时钟
20	Rout	控制寄存器堆输出指定编号寄存器数据
21	Rs/Rt	选择R#: 0=rs字段, 1=rt字段
22	RegDst	选择W#: 0=rt字段, 1=rd字段

7条典型MIPS 32指令 (表6.2):

序号	指令	类型	RTL功能说明
1	lw rt,imm(rs)	I型	$R[rt] \leftarrow M[R[rs] + \text{SignExt}(imm)]$
2	sw rt,imm(rs)	I型	$M[R[rs] + \text{SignExt}(imm)] \leftarrow R[rt]$
3	beq rs,rt,imm	I型	If($R[rs] == R[rt]$) $PC \leftarrow PC + 4 + \text{SignExt}(imm) \ll 2$
4	addi rt,rs,imm	I型	$R[rt] \leftarrow R[rs] + \text{SignExt}(imm)$
5	add rd,rs,rt	R型	$R[rd] \leftarrow R[rs] + R[rt]$
6	slt rd,rs,rt	R型	$R[rd] \leftarrow (R[rs] < R[rt]) ? 1 : 0$
7	j imm26	J型	$PC \leftarrow \{(PC+4)_{31:28}, \text{imm}_{26} \ll 2\}$

取指周期通用流程：

1. $M[PC] \rightarrow IR$: 以PC为地址取指令送入IR
2. $PC + 4 \rightarrow PC$: 修改PC (MIPS 32为32位定长指令)
3. 指令译码

1、lw指令执行流程 (3个机器周期)：

图片信息：PPT第25-30页，lw指令格式（图6.9）及操作流程表（表6.3）

周期	节拍	操作	控制信号
取指周期 Mif	T1	$PC \rightarrow AR; PC \rightarrow X$	$PC_{out}=AR_{in}=X_{in}=1$
	T2	$X+4 \rightarrow Z$	$+4=1$
	T3	$Z \rightarrow PC; M[AR] \rightarrow DR$	$Z_{out}=PC_{in}=1; Read=DRE_{in}=1$
	T4	$DR \rightarrow IR$	$DR_{out}=IR_{in}=1$
计算周期 Mcal	T1	$R[rs] \rightarrow X$	$R_{out}=X_{in}=1$
	T2	$IR(I) + X \rightarrow Z$	$IR(I)_{out}=ADD=1$
执行周期 Mex	T1	$Z \rightarrow AR$	$Z_{out}=AR_{in}=1$
	T2	$M[AR] \rightarrow DR$	$Read=DRE_{in}=1$
	T3	$DR \rightarrow R[rt]$	$DR_{out}=R_{in}=1$

- 数据通道：取指 $PC \rightarrow AR \rightarrow MEM \rightarrow DR \rightarrow IR$ ；修改PC $PC \rightarrow X \rightarrow ALU \rightarrow Z \rightarrow PC$ ；计算地址 $R[rs] \rightarrow X \rightarrow ALU$ ； $IR(I) \rightarrow ALU \rightarrow Z$ ；执行 $Z \rightarrow AR \rightarrow MEM \rightarrow DR \rightarrow R[rt]$

2、sw指令执行流程 (3个机器周期)：

图片信息：PPT第31页，sw指令格式（图6.10）及操作流程表（表6.4）

周期	节拍	操作	控制信号
取指周期 Mif	T1~T4	(与lw相同)	—
计算周期 Mcal	T1~T2	(与lw相同)	—
执行周期 Mex	T1	Z -> AR	Zout=ARin=1
	T2	R[rt] -> DR	Rout=Rs/Rt=DRin=1
	T3	DR -> M[AR]	DREout=Write=1

3、beq指令执行流程（3个机器周期）：

图片信息：PPT第35页，beq指令格式（图6.11）及操作流程表（表6.5）

周期	节拍	操作	控制信号
取指周期 Mif	T1~T4	(与lw相同)	—
计算周期 Mcal	T1	R[rs] -> X	Rout=Xin=1
	T2	X - R[rt] -> PSW	Rout=Rs/Rt=SUB=PSWin=1
执行周期 Mex	T1	PC -> X	PCout=Xin=1
	T2	IR(A) + X -> Z	IR(A)out=ADD=1
	T3	if(PSW.equal) Z -> PC	Zout=1; PCin=PSW.equal

4、addi指令执行流程（2个机器周期）：

图片信息：PPT第39页，addi指令格式（图6.12）及操作流程表（表6.6）

周期	节拍	操作	控制信号
取指周期 Mif	T1~T4	(通用取指)	—
执行周期 Mex	T1	R[rs] -> X	Rout=Xin=1
	T2	IR(I) + X -> Z	IR(I)out=ADD=1
	T3	Z -> R[rt]	Zout=Rin=1

5、add指令执行流程（2个机器周期）：

图片信息：PPT第42页，add指令格式（图6.13）及操作流程表（表6.7）

周期	节拍	操作	控制信号
取指周期 Mif	T1~T4	(通用取指)	—
执行周期 Mex	T1	R[rs] -> X	Rout=Xin=1

周期	节拍	操作	控制信号
	T2	$R[rt] + X \rightarrow Z$	$R_{out}=R_s/R_t=ADD=1$
	T3	$Z \rightarrow R[rd]$	$Z_{out}=RegDst=Rin=1$

单总线结构性能分析：

图片信息：PPT第45页，图6.14 单总线结构数据通路指令方框图

$$T_{min_clk} = T_{clk_to_q} + \max(T_{alu}, T_{mem}) + T_{setup}$$

例6.1：

$$CPI = 0.25 \times 9 + 0.1 \times 9 + 0.11 \times 9 + 0.02 \times 7 + 0.52 \times 7 = 7.92$$

$$T_{min_clk} = T_{clk_to_q} + \max(T_{alu}, T_{mem}) + T_{setup} = 30 + \max(200, 250) + 20 = 300ps$$

$$T_{max_freq} = \frac{1}{300ps} = 3.33GHz$$

$$T_{total} = 1000 \times 10^8 \times 7.92 \times 300 \times 10^{-12} = 237.6s$$

6.3.3 专用通路结构的数据通路

专用通路特点：各寄存器之间或寄存器与ALU之间基于专用数据传输通路连接，非总线共享，各通路数据可并行传输。

1、数据通路基本功能部件：

部件	实现	功能
程序计数器PC	32位寄存器	顺序执行时 $PC = PC + 4$
指令存储器	ROM	32位地址输入，32位指令字输出
数据存储器	RAM	32位地址/数据，WE读写控制（0=读，1=写）
寄存器堆	32个32位寄存器	2个读出口、1个写入口
多路选择器	2路/4路/8路	选择输入信号
符号扩展单元	—	16位扩展为32位
移位运算器	—	如 $<<2$ 左移2位
加法器	—	$PC+4$ 、分支目标地址计算
ALU	—	算术逻辑运算，由AluOp决定

图片信息：PPT第49-50页，图6.15 数据通路中的存储部件，图6.16 组合逻辑部件

2、单周期处理器典型数据通路：

图片信息：PPT第51页，图6.25 单周期MIPS处理器的数据通路

- **单周期处理器**：每条指令在一个时钟周期内执行完成（CPI = 1）
- 时钟周期由执行速度最慢的指令决定（木桶原理）
- 指令和数据分别存放在两个存储器中（指令存储器、数据存储器）
- 需2个加法器、5个多路选择器

各类指令数据通路：

图片信息：PPT第52页，图6.17 取指令数据通路

图片信息：PPT第53页，图6.18 R型运算类指令数据通路

图片信息：PPT第54页，图6.19 lw指令数据通路

图片信息：PPT第55页，图6.20 sw指令数据通路

图片信息：PPT第56页，图6.21 支持R型和访存指令的混合数据通路

图片信息：PPT第57页，图6.22 beq指令数据通路

图片信息：PPT第58页，图6.24 j指令数据通路

混合数据通路中的3个多路选择器：

多路选择器	控制信号	功能
第一个	RegDst	0=W#=rt (lw), 1=W#=rd (R型)
第二个	AluSrc	0=SrcB=R2 (R型), 1=SrcB=SignImm (lw/sw)
第三个	MemToReg	0=WD=ALUResult (R型), 1=WD=ReadData (lw)

单周期处理器控制信号（表6.9）：

控制信号	分类	功能说明
Jump	译码	j、jal等无条件分支指令时生成
Branch	译码	beq、bne等条件分支指令
RegDst	多路选择	1=写入rd, 0=写入rt
AluSrc	多路选择	0=ALU输入R2, 1=输入扩展后的立即数
MemToReg	多路选择	1=存储器数据写回寄存器, 0=ALU结果写回
RegWrite	功能控制	1=寄存器堆写操作
AluOp	功能控制	控制ALU不同运算
MemWrite	功能控制	1=数据存储器写操作

单周期处理器性能分析：

$$T_{min_clk} = T_{clk_to_q} + T_{mem} + \max(T_{RF_read}, T_{ext} + T_{mux}) + T_{alu} + T_{mem} + T_{mux} + T_{setup}$$

通常 $T_{RF_read} > T_{ext} + T_{mux}$ ，因此：

$$T_{min_clk} = T_{clk_to_q} + 2T_{mem} + T_{RF_read} + T_{alu} + T_{mux} + T_{setup}$$

例6.2:

$$T_{min_clk} = 30 + 2 \times 250 + 150 + 200 + 25 + 20 = 925\text{ps}$$

$$T_{max_freq} = \frac{1}{925\text{ps}} = 1.08\text{GHz}$$

$$T_{total} = 1000 \times 10^8 \times 1 \times 925 \times 10^{-12} = 92.5\text{s}$$

单周期处理器三大问题:

- 1. 时钟周期长，程序执行效率不高
- 2. 硬件实现效率不高（如增设2个加法器占用晶圆面积）
- 3. 指令存储器和数据存储器分离与实际存储系统不符

3、多周期处理器典型数据通路:

图片信息: PPT第65-66页，图6.27 多周期MIPS处理器的数据通路

多周期与单周期的区别:

特性	多周期处理器	单周期处理器
存储器	指令和数据保存在同一存储器	指令/数据存储器分离
功能单元复用	ALU、寄存器堆可多次使用	资源专用，不可复用
附加寄存器	增加5个（IR、DR、A、B、C）	无
多路选择器	增加6个	5个
加法器	1个（ALU兼任）	2个

多周期处理器5个附加寄存器:

寄存器	功能	写使能控制
IR	暂存从存储器读出的指令	IrWrite
DR	暂存从存储器读出的数据	无（自动锁存）
A	暂存从寄存器堆读出的数据	无
B	暂存从寄存器堆读出的数据	无
C	暂存ALU运算结果	无

6个多路选择器:

多路选择器	功能
存储器前	区分指令地址和数据地址

多路选择器	功能
寄存器堆W#	确定W#是rt还是rd
寄存器堆WD	确定WD来自ALU还是存储器
ALU SrcA	确定ALU输入A来自PC还是寄存器A
ALU SrcB (4选1)	确定ALU输入B: B值/常数4/立即数/偏移地址
ALU后	确定PC输入: PC+4还是分支目标地址

多周期处理器控制信号 (表6.10):

控制信号	功能说明
Branch	分支指令译码信号
RegDst	1=写入rd, 0=写入rt
AluSrcA	0=输入PC, 1=输入寄存器A
AluSrcB	00=B, 01=常数4, 10=立即数, 11=偏移地址
AluCtrl	0=ALU做加法, 1=由OP和funct决定
MemToReg	0=C寄存器结果, 1=DR寄存器值
PCSrc	0=PC+4, 1=分支目标地址
IorD	0=指令地址, 1=数据地址
RegWrite	寄存器堆写使能
MemWrite	存储器写使能
PCWrite	PC使能信号
IRWrite	IR使能信号

多周期处理器指令执行过程:

- 图片信息: PPT第70页, 图6.28 取指令阶段数据通路
- 图片信息: PPT第71页, 图6.29 译码/取数阶段数据通路
- 图片信息: PPT第72页, 图6.30 R型运算执行阶段数据通路
- 图片信息: PPT第73页, 图6.31 I型运算执行阶段数据通路
- 图片信息: PPT第74页, 图6.32 lw指令执行阶段数据通路
- 图片信息: PPT第75页, 图6.33 sw指令执行阶段数据通路
- 图片信息: PPT第76页, 图6.34 beq指令执行阶段数据通路
- 图片信息: PPT第77页, 图6.35 多周期MIPS处理器指令图

指令	时钟周期数	T1	T2	T3	T4	T5
lw	5	取指	译码/取数	地址运算	访存	写回
sw	4	取指	译码/取数	地址运算	写存	—

指令	时钟周期数	T1	T2	T3	T4	T5
beq	3	取指	译码/取数	条件分支	—	—
add	4	取指	译码/取数	数据运算	写回	—
addi	4	取指	译码/取数	数据运算	写回	—

多周期处理器性能分析：

$$T_{min_clk} = T_{clk_to_q} + T_{mux} + \max(T_{alu} + T_{mux}, T_{mem}) + T_{setup}$$

例6.4：

$$CPI = 0.25 \times 5 + 0.1 \times 4 + 0.11 \times 3 + 0.02 \times 3 + 0.52 \times 4 = 4.12$$

$$T_{min_clk} = 30 + 25 + \max(200 + 25, 250) + 20 = 325\text{ps}$$

$$T_{max_freq} = \frac{1}{325\text{ps}} = 3.08\text{GHz}$$

$$T_{total} = 1000 \times 10^8 \times 4.12 \times 325 \times 10^{-12} = 133.9\text{s}$$

三种数据通路性能比较：

指标	单总线	单周期	多周期
CPI	7.92	1	4.12
T_{min_clk}	300ps	925ps	325ps
T_{max_freq}	3.33GHz	1.08GHz	3.08GHz
执行时间	237.6s	92.5s	133.9s

- 性能从优到劣：单周期 > 多周期 > 单总线
- 多周期中的访存指令lw拖慢了时钟周期，lw被分成5个步骤，但最慢步骤进行了时钟同步，时钟周期并不是单周期的1/5

6.4 时序与控制

6.4.1 中央处理器的时序

单周期处理器：所有指令在1个时钟周期内完成；控制信号同时产生，没有先后顺序。

三级时序系统（传统计算机）：

级别	名称	说明
第一级	指令周期	执行一条指令的时间

级别	名称	说明
第二级	机器周期	如取指周期Mif、计算周期Mcal、执行周期Mex
第三级	时钟周期（节拍）	最基本的定时单位

图片信息： PPT第88页，图6.36 三级时序系统示意图

- **状态周期：** 表示当前指令周期处于哪个机器周期
- **节拍电位：** 表示当前机器周期处于哪个节拍
- **节拍脉冲：** 一个节拍电位内设置一个或多个节拍脉冲

现代时序系统： 一个时钟周期就是一个节拍，不再设置节拍脉冲。

6.4.2 控制方式

1、同步控制方式（固定时序方式）

选取各部件中最长的操作时间作为统一的时间间隔标准（时钟周期）。

（1）定长指令周期：

- 每条指令包含的机器周期数固定
- 每个机器周期包含的时钟周期数固定
- 如：1个指令周期 = 3个机器周期 × 4个时钟周期 = 12个时钟周期

图片信息： PPT第89页，图6.37 定长指令周期的三级时序

（2）变长指令周期：

- 不同指令包含的机器周期数不同
- 不同机器周期包含的时钟周期数不同
- 如：lw/sw/beq = 3个机器周期（9个时钟），add/addi = 2个机器周期（7个时钟）

图片信息： PPT第90页，图6.38 变长指令周期的三级时序

2、异步控制方式

- 没有统一时钟信号，没有固定的机器周期和节拍
- 各功能部件采用应答机制：发出操作控制信号后，等待功能部件发出应答信号才进行下一步

3、联合控制方式

- 同步控制和异步控制相结合
- 大多数操作采用同步控制；少数时间难以确定的操作采用异步控制

6.4.3 时序发生器

时序发生器产生状态周期与节拍电位。

输入：时钟信号CLK、指令译码信号、反馈信号（PSW、中断请求等）、启动/停止/复位信号

输出：状态周期、节拍电位

图片信息：PPT第92页，图6.39 时序发生器工作原理

(1) 定长指令周期状态机：

- 状态周期和节拍电位只与时钟CLK有关
- 每条指令12个状态：S0~S3（取指），S4~S7（计算），S8~S11（执行）

图片信息：PPT第93页，图6.40 定长指令周期三级时序状态机

(2) 变长指令周期状态机：

- 状态周期和节拍电位除与CLK有关，还与指令译码、反馈信号有关
- 共9个状态：S0~S3（取指），S4~S5（计算），S6~S8（执行）
- lw/sw/beq有9个状态，add/addi只有7个状态

图片信息：PPT第94页，图6.41 变长指令周期三级时序状态机

图片信息：PPT第95页，表6.12 变长指令周期状态转换表

6.5 硬布线控制器

6.5.1 三级时序硬布线控制器

硬布线控制器也称为组合逻辑控制器，由组合逻辑电路实现，结构复杂但速度较快。RISC处理器通常采用硬布线控制器。

图片信息：PPT第96页，图6.43 传统三级时序硬布线控制器模型

硬布线控制器设计流程：

1. 分析数据通路，画出指令周期流程图，列出各节拍的控制信号
2. 设计时序发生器：构建状态图
3. 找出同一微操作控制信号的产生条件
4. 写出各微操作控制信号的逻辑表达式
5. 利用组合逻辑电路（逻辑门、PLA、ROM）实现

6.5.2 三级时序硬布线控制器设计

1、单总线结构处理器硬布线控制器

以控制信号的逻辑表达式实现为例：

$$Xin = M_{if} \cdot T_1 + M_{cal} \cdot T_1 + M_{ex} \cdot T_1 \cdot (beq + addi + add)$$

$$Zout = M_{if} \cdot T_3 + M_{ex} \cdot T_1 \cdot (lw + sw) + M_{ex} \cdot T_3 \cdot (beq + addi + add)$$

$$PCin = M_{if} \cdot T_3 + M_{ex} \cdot T_3 \cdot beq \cdot equal$$

2、单周期MIPS处理器硬布线控制器

单周期处理器控制信号只与指令译码信号有关：

控制信号	逻辑表达式
Jump	j + jal
Branch	beq + bne
RegDst	add + sub + and
AluSrc	addi + andi
MemToReg	lw
RegWrite	add + sub + and + addi + andi + lw
MemWrite	sw

6.5.3 现代时序硬布线控制器

现代计算机不再使用多级时序系统，一个时钟周期就是一个节拍。

图片信息： PPT第104页，图6.44 现代时序硬布线控制器模型

设计流程：

1. 分析数据通路，画出指令周期流程图，确定各节拍的控制信号
2. 绘制指令执行状态转换图
3. 构建状态转换表
4. 实现FSM组合逻辑
5. 实现硬布线控制器组合逻辑电路

6.5.4 现代时序硬布线控制器设计

1、单总线结构处理器

图片信息： PPT第106页，图6.45 指令执行状态转换图（25个状态）

图片信息： PPT第107页，表6.14 现代时序指令状态转换表

2、多周期MIPS处理器

关联： 例6.5对应多周期MIPS处理器的控制器设计。

图片信息：PPT第108-109页，图6.46 多周期MIPS指令执行状态转换图（12个状态S0~S11）

图片信息：PPT第110页，表6.15 多周期MIPS指令状态转换表

6.6 微程序控制器

6.6.1 微程序控制的基本概念

微程序控制的基本思想：仿照程序设计方法，将实现指令功能所需的控制信号按照一定规则编码成微指令，存放在控制存储器（ROM）中。一条指令对应一段微程序，执行指令就是执行微程序。

关联：微程序控制器采用存储技术和程序设计技术，相比硬布线控制器的硬件时序，是一种软件时序，使复杂控制逻辑得到简化。

1、微命令与微操作

- **微命令/微操作：**控制器发出的控制信号
- **相容性微操作：**能在一个时钟周期内同时执行
- **互斥性微操作：**不能在同一个时钟周期内并行执行

2、微指令与微程序

- **微指令：**在一个时钟周期内，一组实现一定操作功能的相容性微操作
- **微指令包括：**操作控制字段 + 顺序控制字段（判断测试字段 + 下地址字段）

图片信息：PPT第113页，图6.47 微指令格式示例

3、微指令周期

- 取出并执行一条微指令所需的时间
- 通常1个微指令周期对应1个机器周期
- 包含4个节拍：T1取微指令，T2~T4执行微指令

图片信息：PPT第114页，图6.48 微指令周期

6.6.2 微程序控制器组成原理

1、微程序控制器组成：

图片信息：PPT第115页，图6.49 微程序控制器组成框图

部件	功能
控制存储器	存放全部指令的所有微程序，通常采用ROM

部件	功能
微地址寄存器 (μAR)	为控制存储器提供微地址；初始化时 $\mu AR = 0$
地址转移逻辑	产生后续地址（下一条微指令的地址）

2、后续微地址的形成：

方法	说明
下址字段法（断定法）	在微指令中设置专门的下址字段
计数器法	后续微地址由 $\mu AR + 1$ 得到，无下址字段；需增加Pend测试位

图片信息：PPT第117页，图6.49（下址字段法）和图6.50（计数器法）

3、地址转移逻辑：

图片信息：PPT第118页，图6.51 下址字段法地址转移逻辑

图片信息：PPT第119页，图6.52 计数器法地址转移逻辑

图片信息：PPT第120页，图6.53 采用ROM实现的地址转移逻辑

6.6.3 微程序控制器设计

微指令格式：

图片信息：PPT第122页，图6.54（下址字段法）和图6.55（计数器法）

各指令微程序设计：

图片信息：PPT第123页，图6.56 取指微程序

图片信息：PPT第124页，图6.57 lw指令微程序

图片信息：PPT第125页，图6.58 sw指令微程序

图片信息：PPT第126页，图6.59 beq指令微程序

图片信息：PPT第127页，图6.60a add指令微程序

图片信息：PPT第128页，图6.60b addi指令微程序

微程序容量计算：

- 下址字段法：25条微指令 $\times$ 29位 = 25 $\times$ 29位
- 计数器法：25条微指令 $\times$ 25位 = 25 $\times$ 25位

图片信息：PPT第129-130页，图6.61 微程序总览

微程序控制器设计流程：

1. 分析数据通路，列出每条指令的执行操作流程和每一步的控制信号
2. 细化操作流程，将每个微操作分配到各机器周期的节拍上

- 3. 以时钟周期为单位构建指令执行状态图
- 4. 设计微指令格式、微命令编码方式
- 5. 编制每条指令的微程序，组织微程序并存放到控制存储器中
- 6. 构建地址转移逻辑，实现微程序控制器

微程序控制器 vs 硬布线控制器：

特性	微程序控制器	硬布线控制器
性能	较差（频繁访问控存）	较快
设计规整性	规整、实现容易	结构复杂
可修改性	易修改（采用EEPROM/Flash）	不易修改
适用	CISC（x86、IBM S/360、VAX）	RISC（MIPS、ARM、RISC-V）

6.6.4 微指令及其编码方法

1、微命令编码方法：

方法	原理	优点	缺点
直接表示法	每1位对应1个微命令，1=发出，0=不发出	简单、并行能力强、速度快	操作控制字段长
编码表示法（字段译码法）	互斥微操作分组，各字段通过译码器生成信号	缩短微指令字长	译码器降低速度
混合表示法	直接表示法和编码表示法混合	折中方案	—

图片信息：PPT第132-133页，图6.62 三种编码方法对比

例6.6：

图片信息：PPT第134页，图6.63 各字段位数分配

- 最多可表示微命令： $9 + 7 + 7 + 11 = 34$ 个
- 一条微指令中可同时出现的微命令最多： $9 + 1 + 1 + 1 = 12$ 个
- 控制存储器最大容量： $2^5 \times 24 = 32 \times 24$ 位

2、微指令格式：

类型	特点
水平型微指令	一个微指令周期内同时给出多个微命令；面向数据通路的描述
垂直型微指令	类似于机器指令编码，一条微指令控制1~2个微操作；面向操作算法的描述

- 水平型：微程序短、执行效率高、并行能力强
- 垂直型：结构简单规整、字长短、易于编制，但几乎没有并行能力

6.7 异常与中断处理

关联：异常与中断处理是CPU与外部设备交互的关键机制，与第9章（输入输出系统）密切相关。本节内容原本规划并入第9章，但在此做基本概念介绍。

6.7.1 异常与中断的基本概念

异常（Exception）： CPU内部引起的异常事件，也称内部中断、软件中断。

异常类型	说明	示例
故障（Fault）	指令执行引起的异常	未定义指令、缺页、除零、溢出等
自陷（Trap）	事先安排的"异常"事件	系统调用（INT 21H、syscall）
终止（Abort）	随机出现的硬件故障	机器校验错、总线错误

中断（Interrupt）： 由外部设备向CPU发出的中断请求（如鼠标点击、按键动作）。

术语差异：

- MIPS处理器：将异常和中断都称为"异常"
- x86处理器：将异常和中断都称为"中断"，来自内部的称"内部中断"，来自外部的称"外部中断"

6.7.2 异常与中断处理过程

中断响应周期的三大任务：

任务	说明
关中断	临时禁止新的中断请求，保护现场操作的完整性
保存断点	保存返回被中断程序的位置（下一条指令或异常指令的PC值）
中断识别	识别中断来源，将中断服务程序入口地址送入PC

- 中断响应周期由硬件实现，不可被打断
- 中断服务程序由软件实现
- 整个中断处理是软硬件协同实现的

6.7.3 支持中断的CPU设计

1、数据通路升级：

新增部件	功能	控制信号
EPC	异常程序计数器，保存断点	EPCin、EPCout
IE	中断使能位	STI（开中断）、CLI（关中断）
中断控制逻辑	中断优先级排队	ClrInt
多路选择器MUX	根据中断号选择服务程序入口	IntAout

图片信息：PPT第140页，图6.64 支持中断的单总线结构计算机框图

图片信息：PPT第140页，表6.18 中断控制信号及其作用

2、操作控制器升级：

(1) 三级时序硬布线控制器升级：

图片信息：PPT第141页，图6.65 支持中断的三级时序状态机

- 增加中断响应周期Mint（2个状态S9、S10）
- S9：关中断+保存断点（CLI=1, PCout=1, EPCin=1）
- S10：中断识别（IntAout=1, PCin=1）
- 增加中断返回指令eret

(2) 现代时序硬布线控制器升级：

图片信息：PPT第142页，图6.66 支持中断的现代时序状态机

- 增加S26、S27用于中断响应
- 增加eret指令执行周期S25

(3) 微程序控制器升级：

图片信息：PPT第143页，图6.67 支持中断的微指令格式

图片信息：PPT第143页，图6.68 支持中断的微程序控制器原理图

- 增加6个中断控制信号
- 增加2条中断响应微指令
- 增加1条eret微指令
- 需要Pend测试位进行中断判断

3、中断服务程序：

1. 保护现场：PUSH AX；PUSH BX；PUSH CX
2. 中断服务：完成中断处理（核心内容）

3. 恢复现场: POP CX; POP BX; POP AX
4. 中断返回: **eret** (返回到断点处)

实践训练 (实验5)

1. 用Logisim构建单总线结构CPU, 支持MIPS32指令, 运行冒泡排序程序
 - 传统三级时序硬布线控制器
 - 现代时序硬布线控制器
 - 微程序控制器
2. 为单总线CPU增加简单的中断处理逻辑 (按键中断)
3. 用Logisim构建单周期MIPS处理器
4. 用Logisim构建多周期MIPS处理器 (硬布线或微程序控制器)